portal needs to have access to the private key used to sign the JWTs, while each API server needs access to only the public key. Using public key-signed JWTs in this way is covered in section 7.4.4 of chapter 7, and the same approach can be used here, with a developer portal taking the place of the authorization server (AS).

> **CAUTION** Although using JWTs for client authentication is more secure than client secrets, a signed JWT is still a bearer credential that can be used by anybody that captures it until it expires. A malicious or compromised API server could take the JWT and replay it to other APIs to impersonate the client. Use expiry, audience, and other standard JWT claims (chapter 6) to reduce the impact if a JWT is compromised.

11.2 The OAuth2 client credentials grant

Although JWT bearer authentication is appealing due to its apparent simplicity, you still need to develop the portal for generating JWTs and you'll need to consider how to revoke tokens when a service is retired, or a business partnership is terminated. The need to handle service-to-service API clients was anticipated by the authors of the OAuth2 specifications, and a dedicated grant type was added to support this case: the client credentials grant. This grant type allows an OAuth2 client to obtain an access token using its own credentials without a user being involved. The access token issued by the authorization AS can be used